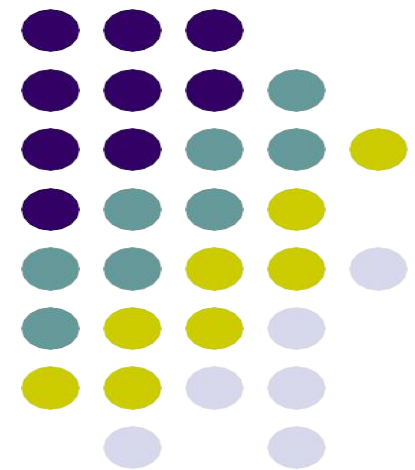




Operating Systems

Sidharth Sharma
Spring 2025



Grading Policy



Endsem: 50%

In-video quiz: 15%

Per week, two hours: TAs: Live session quiz: 10%

Lab: 25%

Recorded lecture interaction, doubt clearing, or problem solving

General Information



Textbook

OS Part:

1. **Operating System Concepts**, 8th Ed, by Silberschatz, Galvin, and Gagne
2. **Design of the Unix Operating System**, by Maurice Bach
3. **Operating Systems: Three Easy Pieces**, Remzi H. Arpaci-Dusseau and Andrea C. Arpaci-dusseau

What is an Operating System?



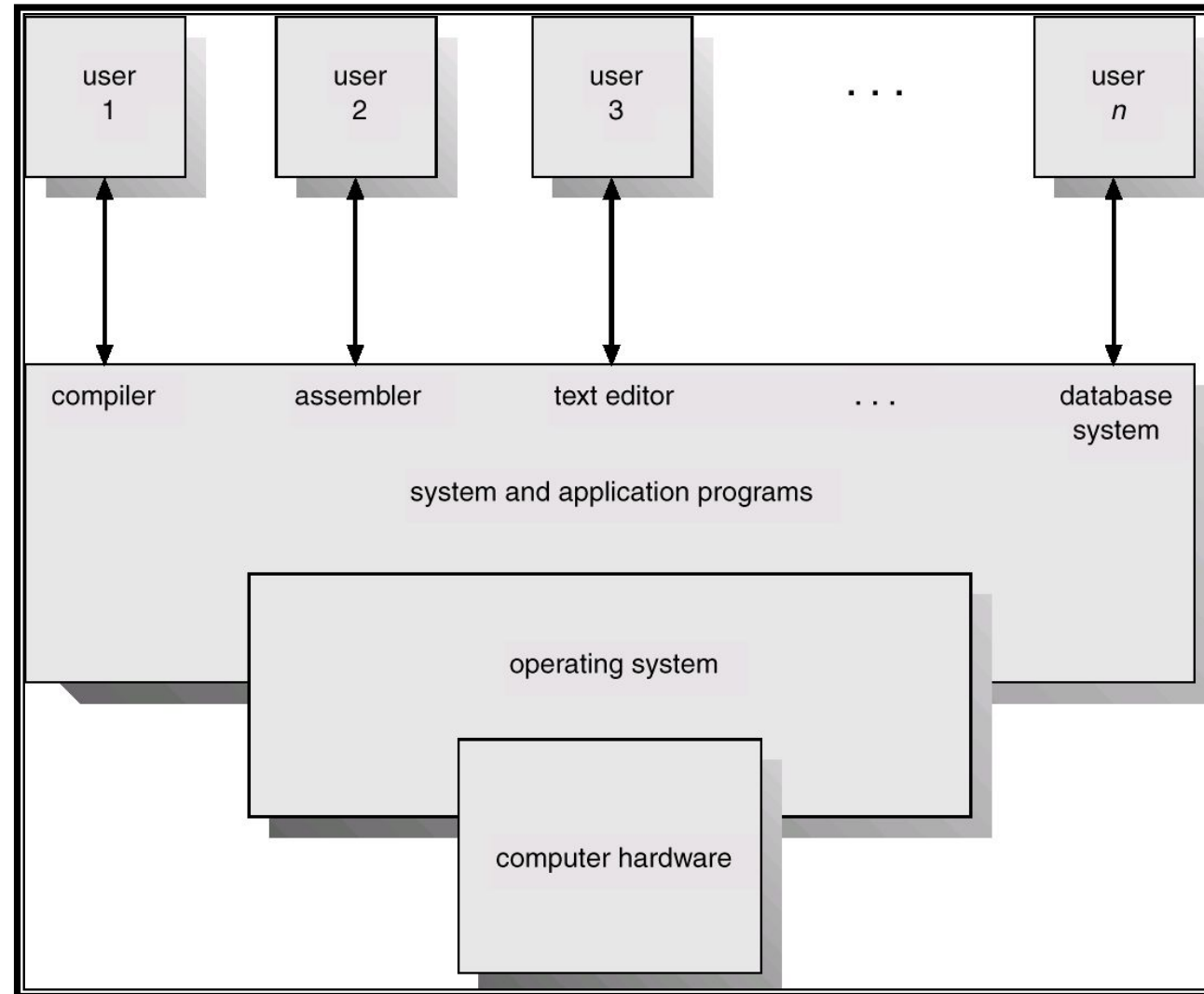
- User-centric definition
 - A program that acts as an intermediary between a user of a computer and the computer hardware
 - Defines an interface for the user to use services provided by the system
 - Provides a “view” of the system to the user
- System-centric definition
 - Resource allocator – manages and allocates resources
 - Control program – controls the execution of user programs and operations of I/O devices

Computer System Structure



- The computer system can be divided into four components:
 - Hardware – provides basic computing resources
 - 4 CPU, memory, I/O devices
 - Operating system
 - 4 Controls and coordinates the use of the hardware among various applications and users
 - Application programs – define the ways in which the system resources are used to solve the computing problems of the users
 - 4 Word processors, compilers, web browsers, database systems, video games
 - Users
 - 4 People, machines, other computers

Abstract View of System Components



What Operating Systems Do



- Depends on the point of view
- Users want convenience, **ease of use** and **good performance**
 - Don't care about **resource utilization**
- But shared computers such as **mainframe** or **minicomputer** must keep all users happy
- Handheld computers are resource-poor, optimized for usability and battery life
- Some computers have little or no user interface, such as embedded computers in devices and automobiles

Computer Startup

bootstrap program is loaded at power-up or reboot

- Typically stored in ROM or EPROM, generally known as **firmware**
- Initializes all aspects of the system
- Loads operating system kernel and starts execution

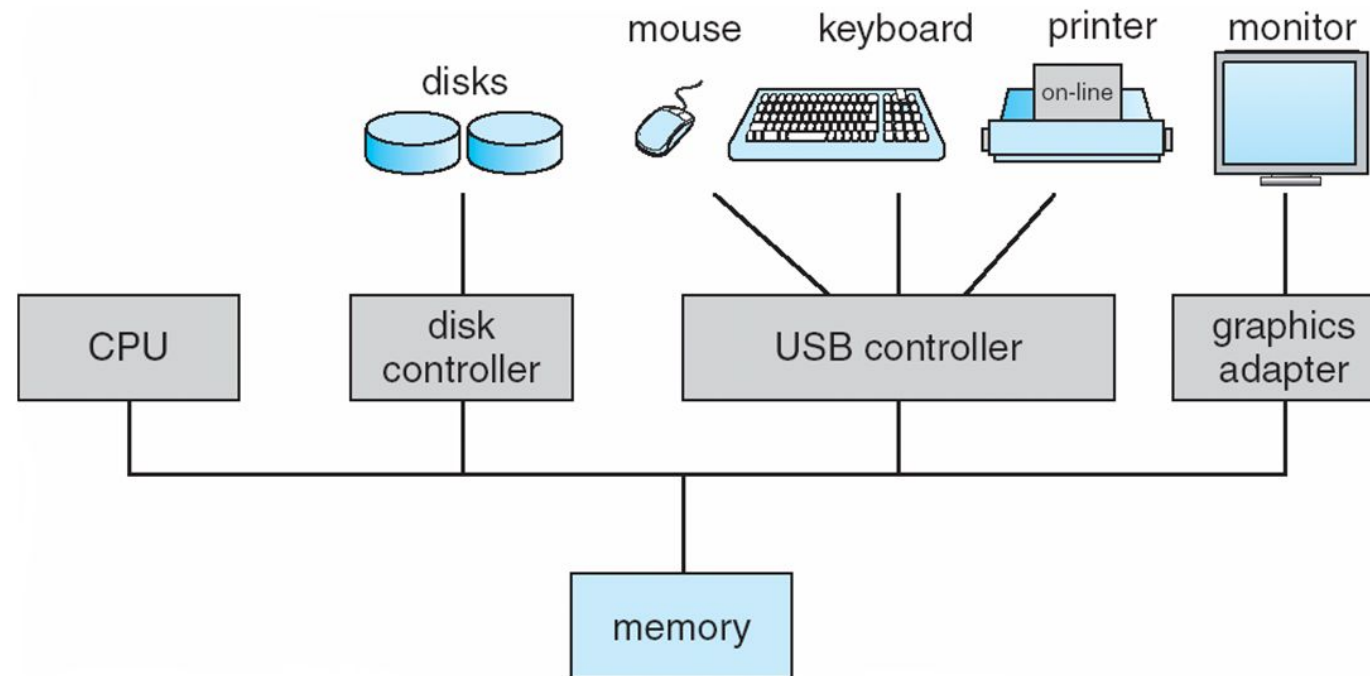


Computer System Organization



Computer-system operation

One or more CPUs, device controllers connect through common bus providing access to shared memory
Concurrent execution of CPUs and devices competing for memory cycles



Computer-System Operation



- I/O devices and the CPU can execute concurrently
- Each device controller is in charge of a particular device type
- Each device controller has a local buffer
- CPU moves data from/to main memory to/from local buffers
- Device controller informs CPU that it has finished its operation by causing an **interrupt**

Storage Hierarchy



Storage systems organized in hierarchy

- Speed

- Cost

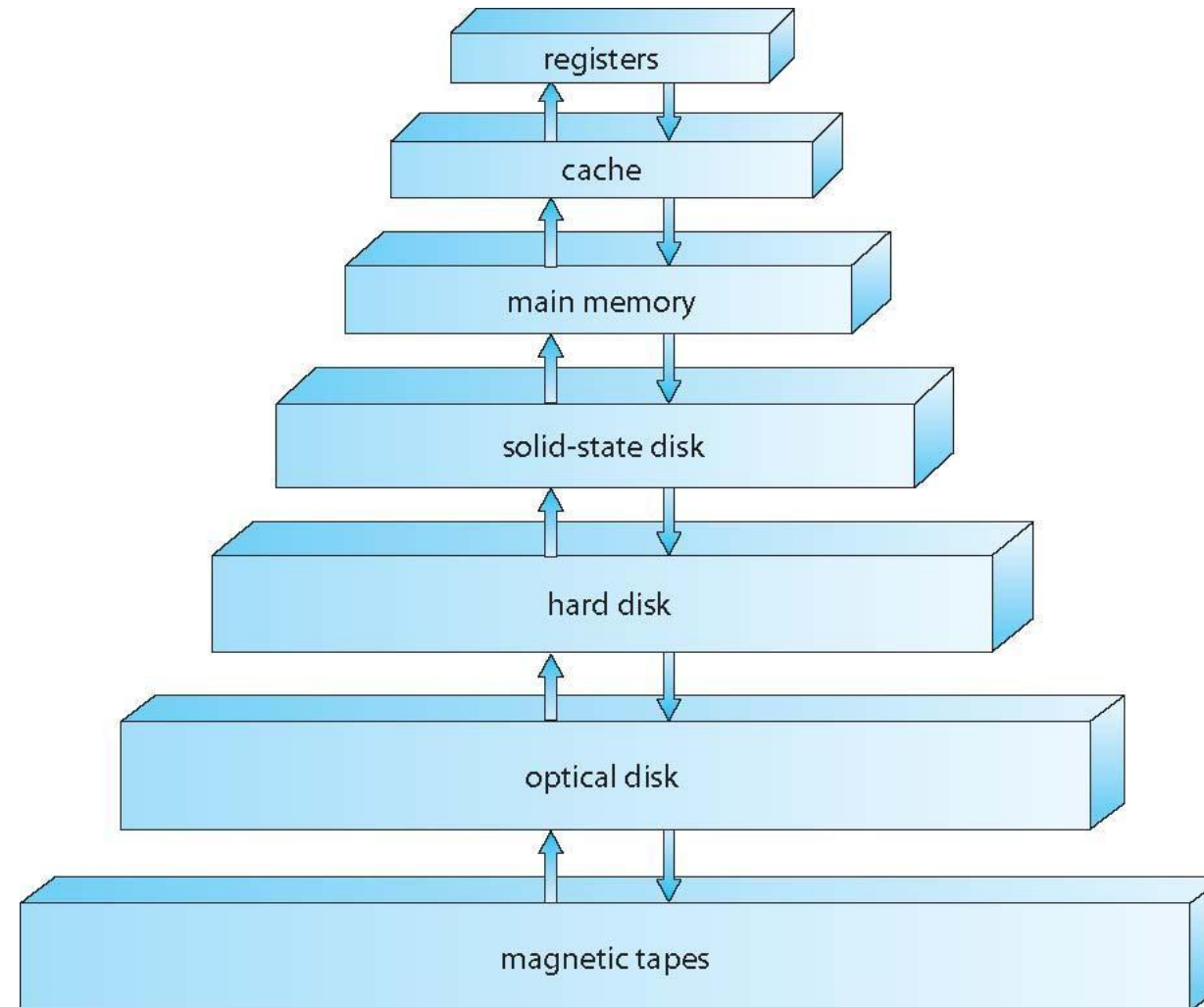
- Volatility

Caching – copying information into faster storage system; main memory can be viewed as a cache for secondary storage

Device Driver for each device controller to manage I/O

- Provides uniform interface between controller and kernel

Storage-Device Hierarchy

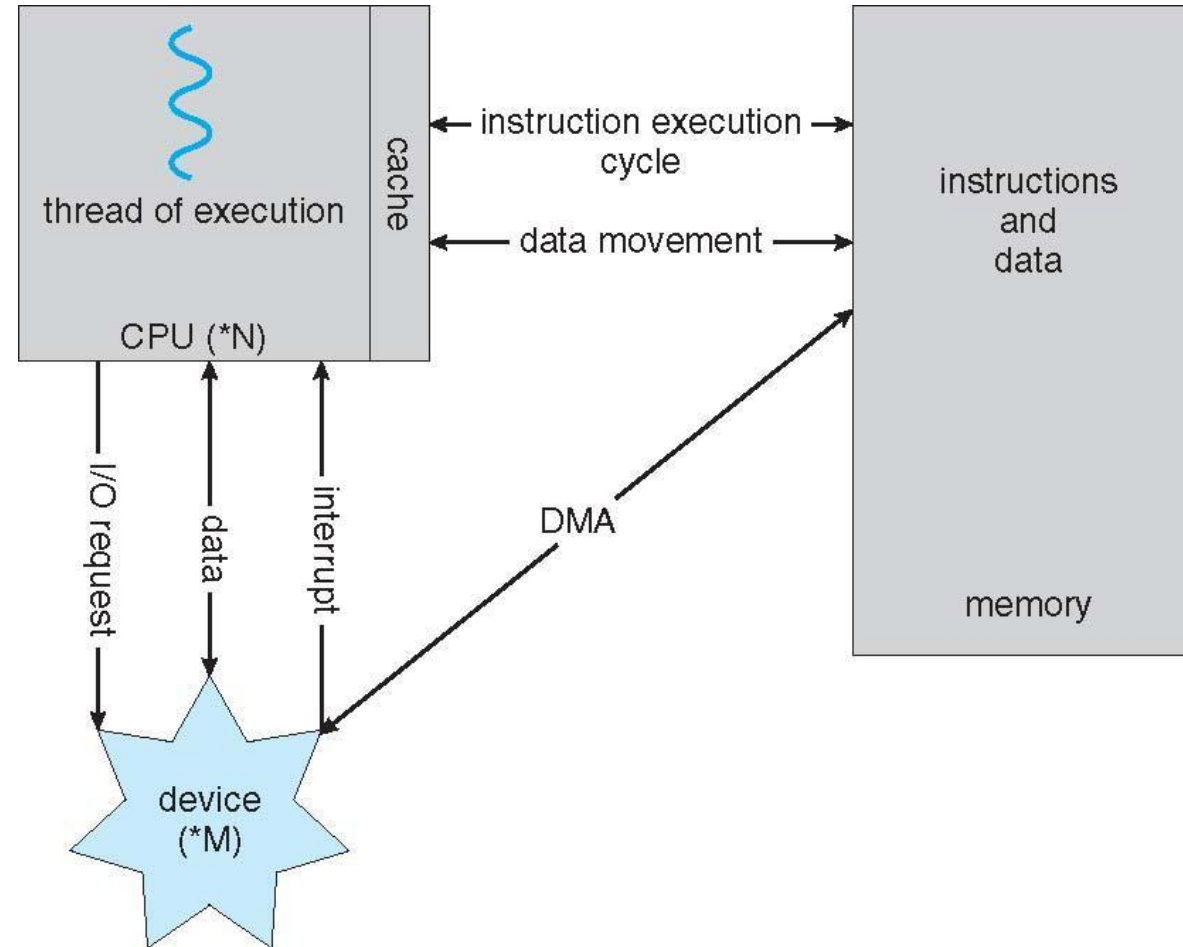


Caching



- Important principle, performed at many levels in a computer (in hardware, operating system, software)
- Information in use copied from slower to faster storage temporarily
- Faster storage (cache) checked first to determine if information is there
 - If it is, information used directly from the cache (fast)
 - If not, data copied to cache and used there
- Cache smaller than storage being cached
 - Cache management important design problem
 - Cache size and replacement policy

How a Modern Computer Works



A von Neumann architecture

Operating System Structure



Multiprogramming (Batch system) needed for efficiency

- Single user cannot keep CPU and I/O devices busy at all times
- Multiprogramming organizes jobs (code and data) so CPU always has one to execute
- A subset of total jobs in system is kept in memory
- One job selected and run via **job scheduling**
- When it has to wait (for I/O for example), OS switches to another job

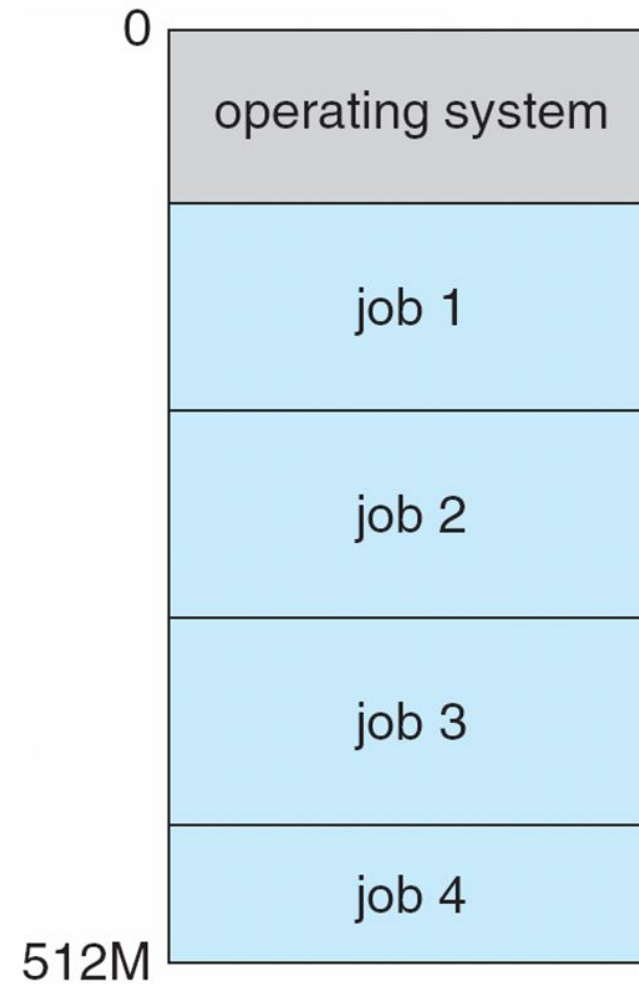
Timesharing (multitasking) is logical extension in which CPU switches jobs so frequently that users can interact with each job while it is running, creating **interactive** computing

- **Response time** should be < 1 second
- Each user has at least one program executing in memory

□ **process**

If several jobs ready to run at the same time □ **CPU scheduling**

Memory Layout for Multiprogrammed System



Operating-System Operations



Interrupt-driven (hardware and software)

- Hardware interrupt by one of the devices
- Software interrupt (**exception** or **trap**):
 - Software error (e.g., division by zero)
 - Request for operating system service
 - Other process problems include infinite loop, processes modifying each other or the operating system

Operating-System Operations (cont.)



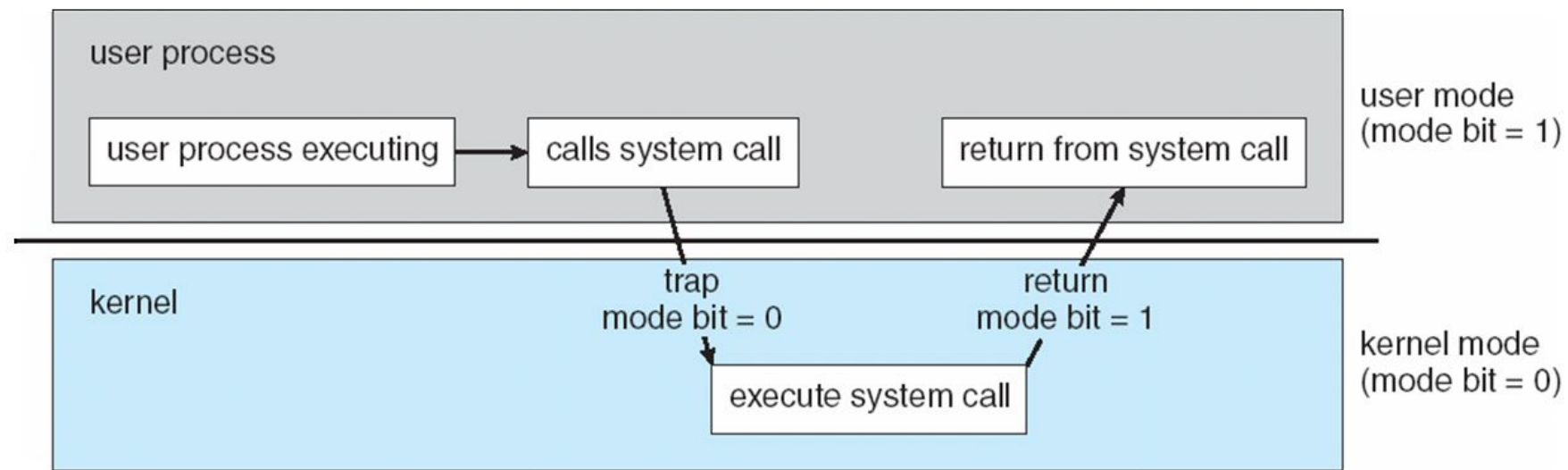
Dual-mode operation allows OS to protect itself and other system components

User mode and **kernel mode**

Mode bit provided by hardware

- Provides ability to distinguish when system is running user code or kernel code
- Some instructions designated as **privileged**, only executable in kernel mode
- System call changes mode to kernel, return from call resets it to user

Transition from User to Kernel Mode



Tasks of Operating Systems



- Process Management
- Memory Management
- I/O Management
- File Management
- Storage Management

Process Management



- A process is a program in execution. It is a unit of work within the system. Program is a ***passive entity***; process is an ***active entity***.
- Process needs resources to accomplish its task
 - CPU, memory, I/O, files
 - Initialization data
- Process termination requires reclaim of any reusable resources
- Typically, system has many processes, some user, some operating system running concurrently on one or more CPUs
 - Concurrency by multiplexing the CPUs among the processes / threads

Process Management Activities



The operating system is responsible for the following activities in connection with process management:

- Creating and deleting both user and system processes
- Suspending and resuming processes
- Providing mechanisms for process synchronization
- Providing mechanisms for process communication
- Providing mechanisms for deadlock handling

Memory Management



- To execute a program all (or part) of the instructions must be in memory
- All (or part) of the data that is needed by the program must be in memory.
- Memory management determines what is in memory and when
 - Optimizing CPU utilization and computer response to users
- Memory management activities
 - Keeping track of which parts of memory are currently being used and by whom
 - Deciding which processes (or parts thereof) and data to move into and out of memory
 - Allocating and deallocating memory space as needed

Storage Management



- OS provides uniform, logical view of information storage
 - Abstracts physical properties to logical storage unit - **file**
 - Each medium is controlled by device (i.e., disk drive, tape drive)
 - Varying properties include access speed, capacity, data-transfer rate, access method (sequential or random)
- File-System management
 - Files usually organized into directories
 - Access control on most systems to determine who can access what
 - OS activities include
 - Creating and deleting files and directories
 - Primitives to manipulate files and directories
 - Mapping files onto secondary storage
 - Backup files onto stable (non-volatile) storage media

Performance of Various Levels of Storage



Level	1	2	3	4	5
Name	registers	cache	main memory	solid state disk	magnetic disk
Typical size	< 1 KB	< 16MB	< 64GB	< 1 TB	< 10 TB
Implementation technology	custom memory with multiple ports CMOS	on-chip or off-chip CMOS SRAM	CMOS SRAM	flash memory	magnetic disk
Access time (ns)	0.25 - 0.5	0.5 - 25	80 - 250	25,000 - 50,000	5,000,000
Bandwidth (MB/sec)	20,000 - 100,000	5,000 - 10,000	1,000 - 5,000	500	20 - 150
Managed by	compiler	hardware	operating system	operating system	operating system
Backed by	cache	main memory	disk	disk	disk or tape

Movement between levels of storage hierarchy can be explicit or implicit

I/O Subsystem



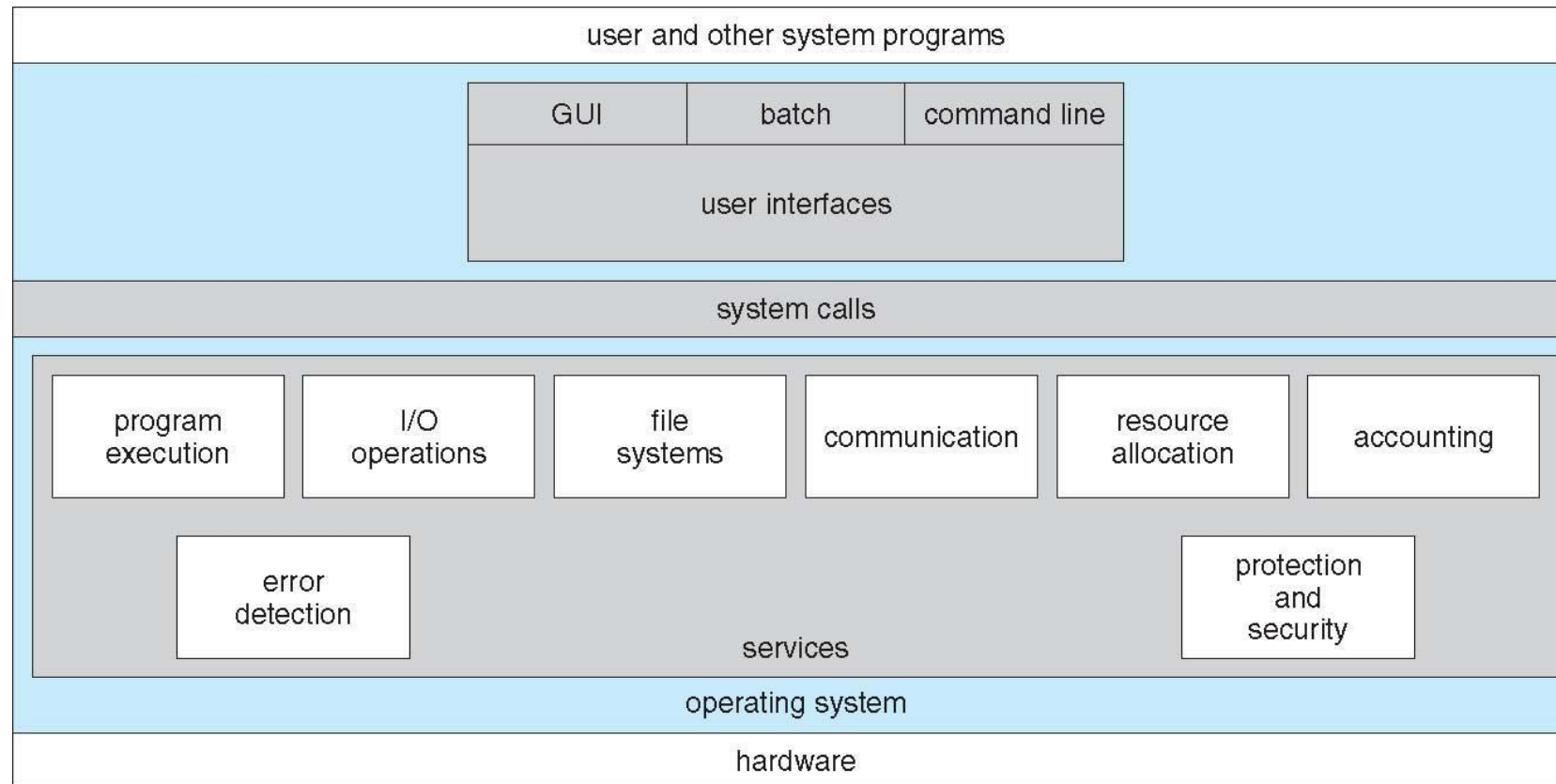
- One purpose of OS is to hide peculiarities of hardware devices from the user
- I/O subsystem responsible for
 - Memory management of I/O including buffering (storing data temporarily while it is being transferred), caching (storing parts of data in faster storage for performance), spooling (the overlapping of output of one job with input of other jobs)
 - General device-driver interface
 - Drivers for specific hardware devices

Protection and Security



- **Protection** – any mechanism for controlling access of processes or users to resources defined by the OS
- **Security** – defense of the system against internal and external attacks
 - Huge range, including denial-of-service, worms, viruses, identity theft, theft of service
- Systems generally first distinguish among users, to determine who can do what
 - User identities (**user IDs**, security IDs) include name and associated number, one per user
 - User ID then associated with all files, processes of that user to determine access control
 - Group identifier (**group ID**) allows set of users to be defined and controls managed, then also associated with each process, file
 - **Privilege escalation** allows user to change to effective ID with more rights

A View of Operating System Services



User Operating System Interface - CLI

- CLI or **command interpreter** allows direct command entry
 - Sometimes implemented in the kernel, sometimes by systems program
 - Sometimes multiple flavors are implemented – **shells**
 - Primarily fetches a command from the user and executes it.



Bourne Shell Command Interpreter



```
Default
New Info Close Execute Bookmarks

PBGMac-Pro:~ pbg$ w
15:24 up 56 mins, 2 users, load averages: 1.51 1.53 1.65
USER      TTY      FROM          LOGIN@  IDLE WHAT
pbg       console -            14:34    50 -
pbg       s000    -            15:05    - w
PBGMac-Pro:~ pbg$ iostat 5
            disk0      disk1      disk10      cpu      load average
      KB/t tps MB/s    KB/t tps MB/s    KB/t tps MB/s  us sy id  1m  5m  15m
      33.75 343 11.30    64.31 14  0.88    39.67  0  0.02  11  5 84  1.51 1.53 1.65
      5.27 320  1.65     0.00  0  0.00     0.00  0  0.00   4  2 94  1.39 1.51 1.65
      4.28 329  1.37     0.00  0  0.00     0.00  0  0.00   5  3 92  1.44 1.51 1.65
^C
PBGMac-Pro:~ pbg$ ls
Applications      Music              WebEx
Applications (Par Pando Packages    config.log
Desktop           Pictures           getsmartdata.txt
Documents         Public            imp
Downloads        Sites             log
Dropbox          Thumbs.db         panda-dist
Library          Virtual Machines  prob.txt
Movies           Volumes           scripts
PBGMac-Pro:~ pbg$ pwd
/Users/pbg
PBGMac-Pro:~ pbg$ ping 192.168.1.1
PING 192.168.1.1 (192.168.1.1): 56 data bytes
64 bytes from 192.168.1.1: icmp_seq=0 ttl=64 time=2.257 ms
64 bytes from 192.168.1.1: icmp_seq=1 ttl=64 time=1.262 ms
^C
--- 192.168.1.1 ping statistics ---
2 packets transmitted, 2 packets received, 0.0% packet loss
round-trip min/avg/max/stddev = 1.262/1.760/2.257/0.498 ms
PBGMac-Pro:~ pbg$
```

User Operating System Interface - GUI



- User-friendly **desktop** metaphor interface
 - Usually mouse, keyboard, and monitor
 - **Icons** represent files, programs, actions, etc
 - Various mouse buttons over objects in the interface cause various actions (provide information, options, execute function, open directory (known as a **folder**))
 - Invented at Xerox PARC
- Many systems now include both CLI and GUI interfaces
 - Microsoft Windows is GUI with CLI “command” shell
 - Apple Mac OS X is “Aqua” GUI interface with UNIX kernel underneath and shells available
 - Unix and Linux have CLI with optional GUI interfaces (CDE, KDE, GNOME)

Touchscreen Interfaces

- Touchscreen devices require new interfaces
 - Mouse not possible or not desired
 - Actions and selection based on gestures
 - Virtual keyboard for text entry
- Voice commands.



System Calls



- Programming interface to the services provided by the OS
- Typically written in a high-level language (C or C++)
- Mostly accessed by programs via a high-level **Application Programming Interface (API)** rather than direct system call use
- The three most common APIs are Win API for Windows, POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and Java API for the Java virtual machine (JVM)

Example of Standard API



EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the `man` page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count)
```

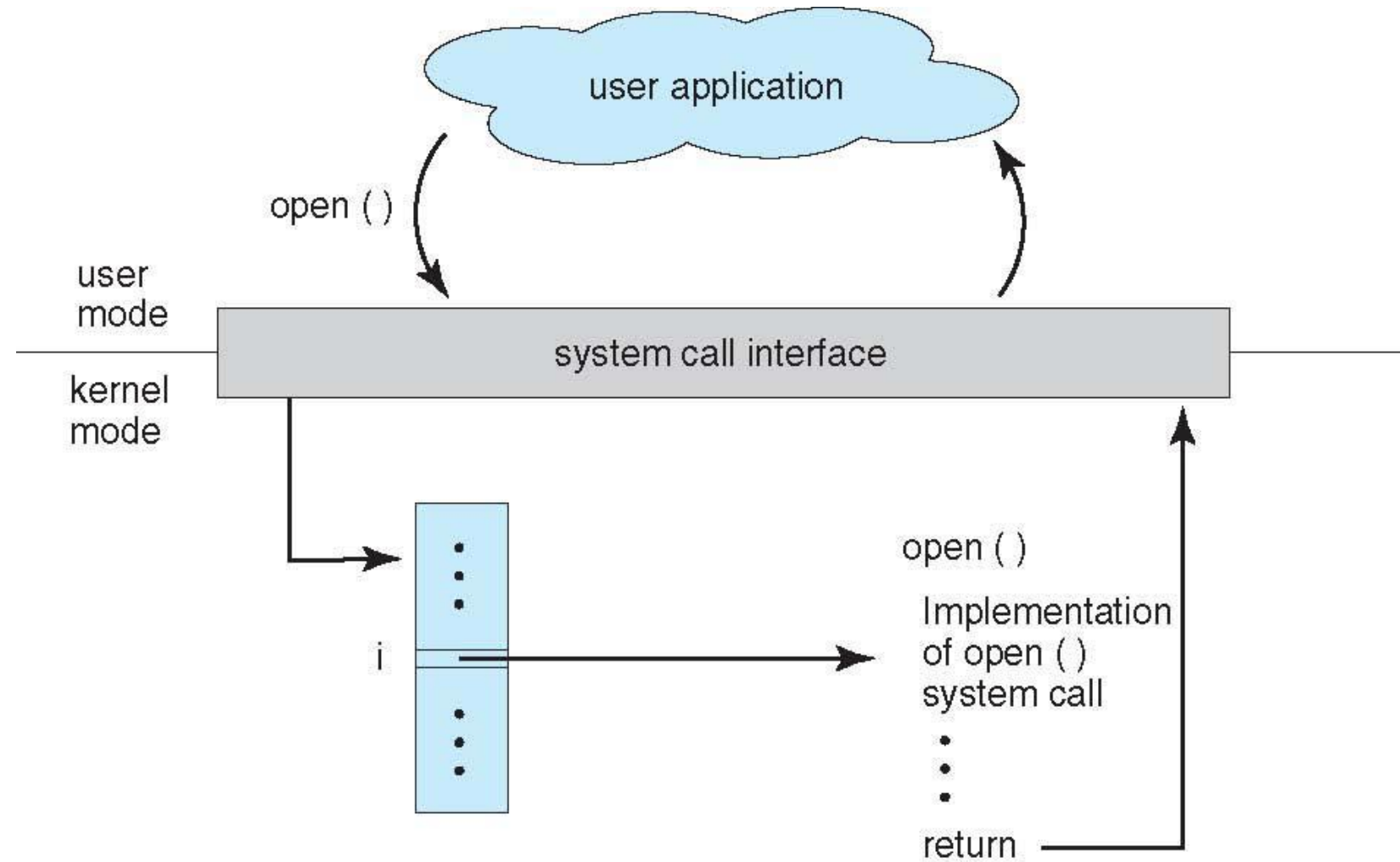
return value	function name	parameters
--------------	---------------	------------

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

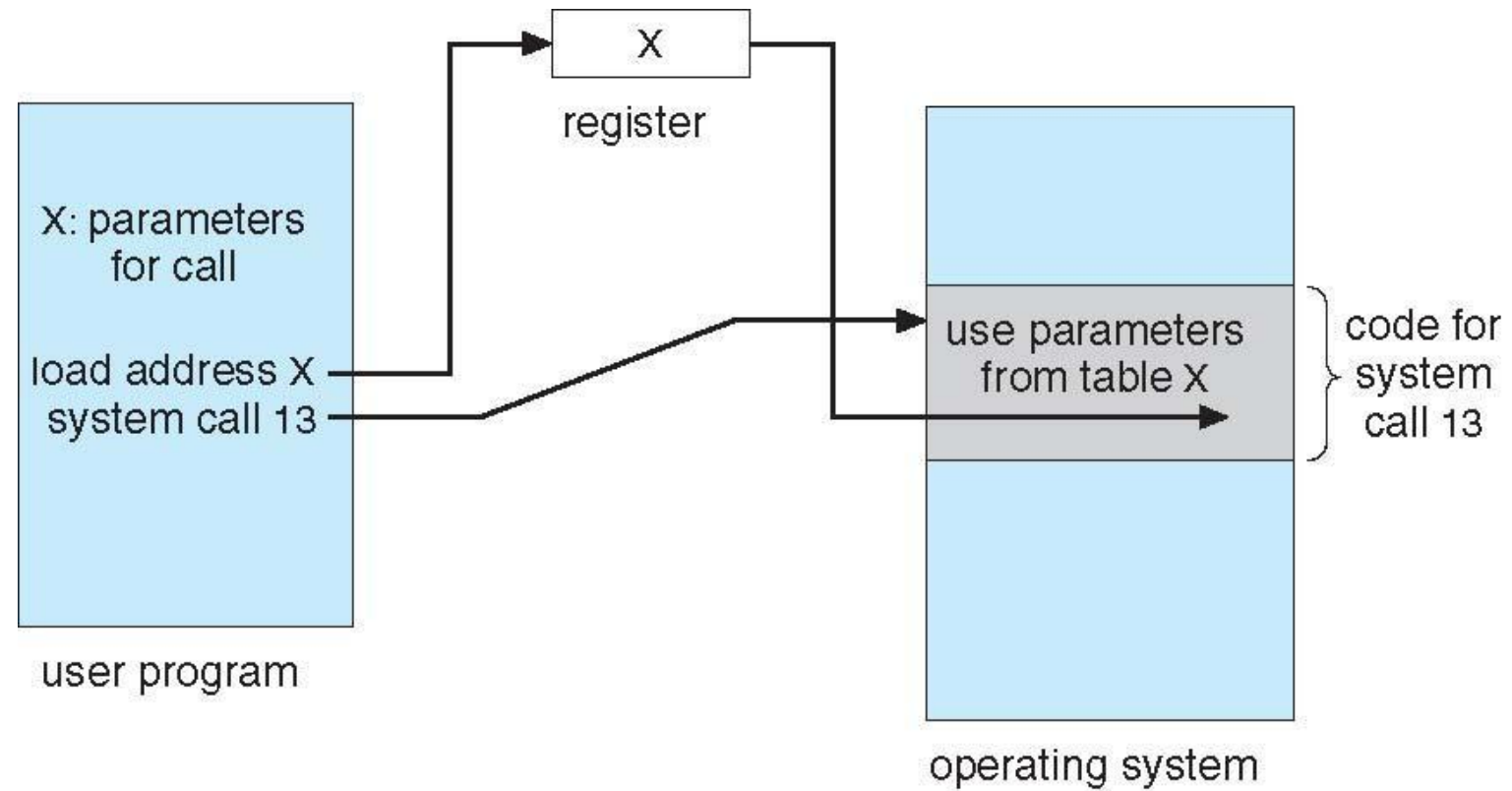
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

API – System Call – OS Relationship



Parameter Passing via Table



Types of System Calls



- Process control
 - create process, terminate process
 - end, abort
 - load, execute
 - get process attributes, set process attributes
 - wait for time
 - wait event, signal event
 - allocate and free memory
 - Dump memory if error
 - **Debugger** for determining **bugs, single step** execution
 - **Locks** for managing access to shared data between processes

Types of System Calls



- File management
 - create file, delete file
 - open, close file
 - read, write, reposition
 - get and set file attributes
- Device management
 - request device, release device
 - read, write, reposition
 - get device attributes, set device attributes
 - logically attach or detach devices

Types of System Calls (Cont.)



- Information maintenance
 - get time or date, set time or date
 - get system data, set system data
 - get and set process, file, or device attributes
- Communications
 - create, delete communication connection
 - send, and receive messages if the **message passing model** to **hostname** or **process name**
 - From **client** to **server**
 - **Shared-memory model** create and gain access to memory regions
 - transfer status information
 - attach and detach remote devices

Types of System Calls (Cont.)



- Protection
 - Control access to resources
 - Get and set permissions
 - Allow and deny user access

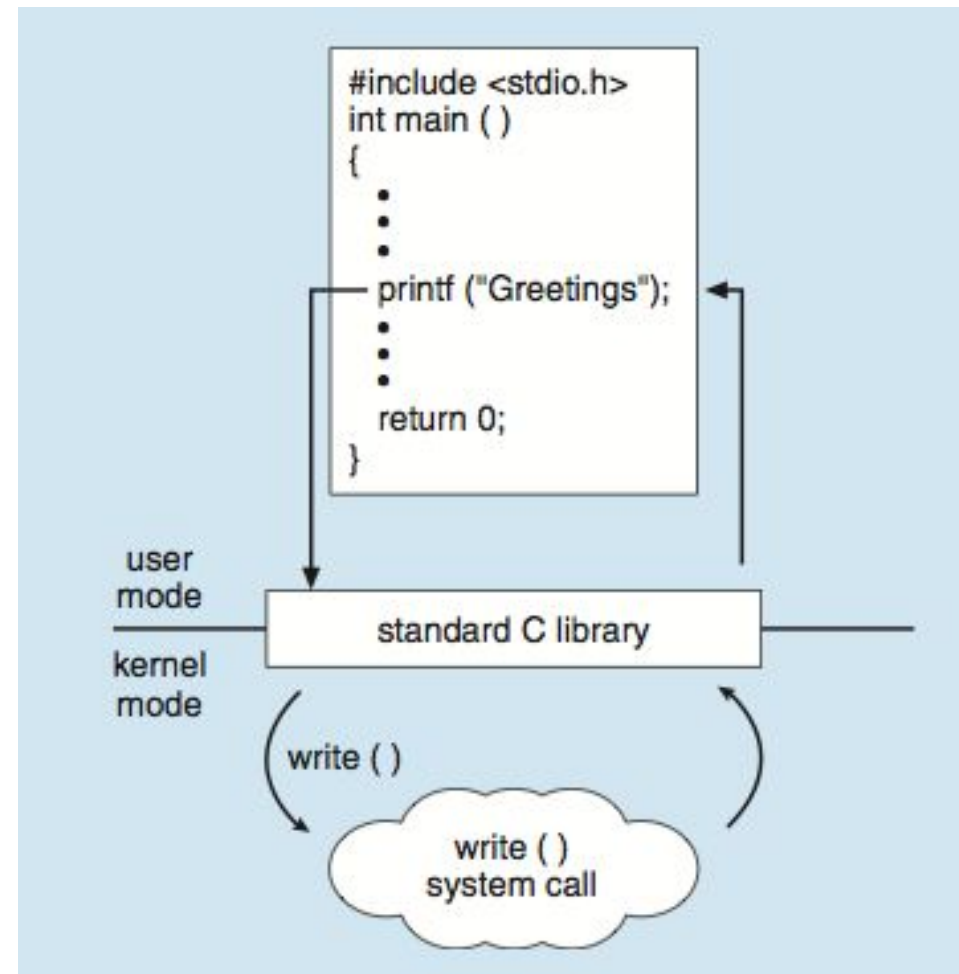
Examples of Windows and Unix System Calls



	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

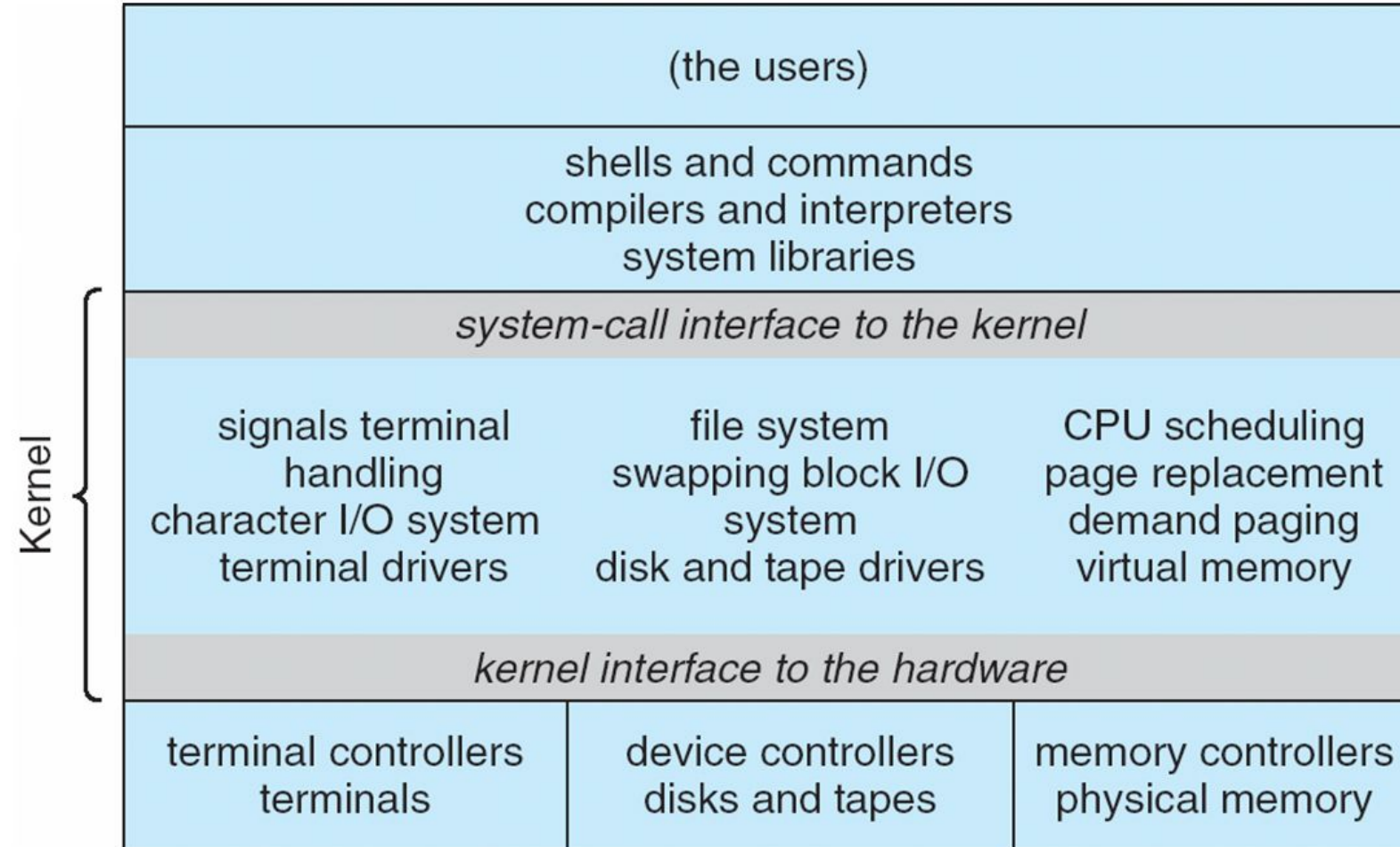
Standard C Library Example

C program invoking printf() library call, which calls write() system call

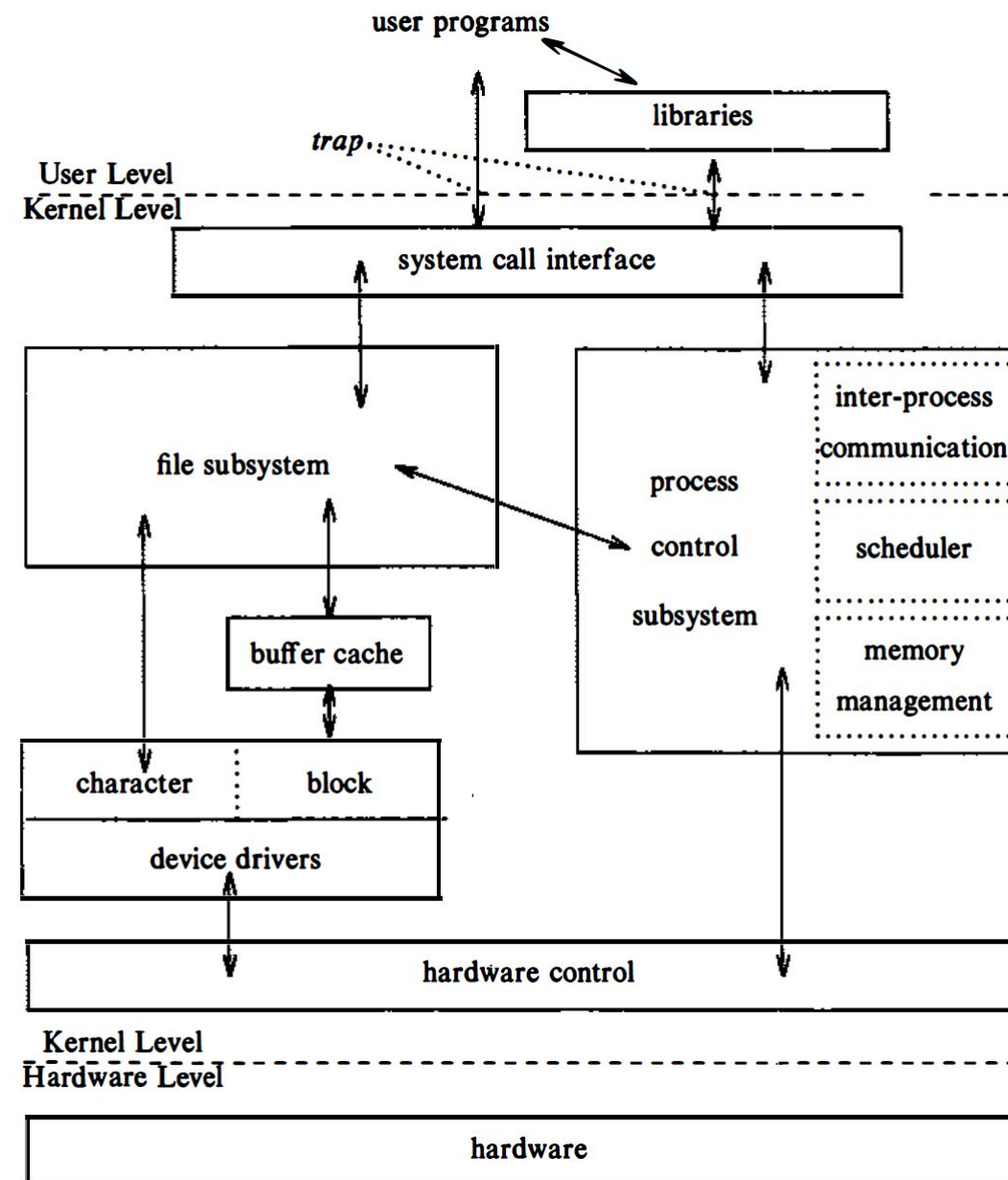


Traditional UNIX System Structure

Beyond simple but not fully layered



Unix Kernel



OS Impact on AI / Data Science Workflows

For AI/DS workloads, the OS is not just background software—it directly affects performance, scalability, and reliability.

Process & Thread Management

Training jobs use multiple processes/threads

- OS scheduler affects:
 - Training time
 - Latency
 - CPU–GPU overlap
- Poor scheduling can lead to underutilized GPUs
- Example: Data loader threads are competing with training threads

Memory Management

AI models require large contiguous memory

OS manages:

- Virtual memory
- Page faults
- Shared memory (e.g., tensors across processes)

Issues:

Excessive swapping → training slowdown

Memory fragmentation → allocation failures

File System & I/O Performance

Unlike traditional programs that read input once:

- Training data is read epoch after epoch
- Same files are accessed thousands of times
- Multiple workers may read the same data concurrently

Examples:

- ImageNet images
 - Large CSV/Parquet files
 - Sharded TFRecords / WebDataset tar files
-
- This makes caching and buffering extremely important.

Impacts:

- Data loading speed
- Training throughput

Example:

- `mmap()`-based dataset loading: `mmap()` (memory mapping) maps a file directly into virtual memory
- Prefetching (Load next batch while current batch is training)

GPU & Accelerator Interaction

OS manages:

- GPU drivers
- DMA transfers
- Context switching

CUDA calls eventually become system calls

Impact:

- Kernel scheduling affects GPU utilization
- NUMA-aware memory placement improves performance

Networking for Distributed Training

Distributed AI relies on:

- Sockets
- RDMA
- TCP/IP or RoCE

OS networking stack affects:

- All-reduce latency
- Parameter synchronization speed

Example:

- Kernel bypass (RDMA) → lower latency than socket

Containers & MLOps

Docker, Kubernetes rely on OS features:

- Namespaces
- cgroups
- Virtual file systems

OS enforces:

- CPU/GPU quotas
- Memory limits
- Isolation between jobs